

Lab 07

Due March 21, 5:00 PM PST

AUTHOR

Brandon Cho - 301365816

Modify this template file to create your submission.

Problem 1a

```
## write your answer here
my_dist <- function(data, centroids) {
  n <- nrow(data)
  K <- nrow(centroids)
  distances <- matrix(NA, n, K)
  for (j in 1:K) {

    distances[, j] <- sqrt(rowSums((data - matrix(centroids[j, ], n, ncol(data), byrow = TRUE))^2
  }
  return(distances)
}
```

Problem 1b

Parameters: data -> the matrix of data to be clustered

k -> number of clusters to create

max_iter -> the maximum number of iterations the algorithm will run

tol -> threshold such that if the all Euclidean distances between the centroids before an iteration and after an iteration is less than this threshold, the iterations stop

Return value:

A list of the centroid positions and a vector that explains which cluster a datapoint is assigned to

```
## write answer here
my_kmeans <- function(data, K, max_iter, tol) {

  data <- as.matrix(data)
  n <- nrow(data)
  d <- ncol(data)
  max_iter <- 100
  tol <- 0.001

  initial_indices <- sample(1:n, K)
  centroids <- data[initial_indices, ]
```

```

cluster_assignments <- rep(0, n)

for (iter in 1:max_iter) {
  # compute distances
  distances <- my_dist(data, centroids)

  # assign clusters
  new_assignments <- apply(distances, 1, which.min)

  # compute mean for each cluster
  new_centroids <- matrix(NA, nrow = K, ncol = d)
  for (k in 1:K) {
    if (sum(new_assignments == k) == 0) {
      # If no points are assigned to cluster k, reinitialize that centroid randomly.
      new_centroids[k, ] <- data[sample(1:n, 1), ]
    } else {
      new_centroids[k, ] <- colMeans(data[new_assignments == k, , drop = FALSE])
    }
  }

  # compute shifts for each centroid
  centroid_shifts <- sqrt(rowSums((centroids - new_centroids)^2))

  # if shift is less the tol, stop loop
  if (all(centroid_shifts < tol)) {
    cat("Converged in", iter, "iterations.\n")
    centroids <- new_centroids
    cluster_assignments <- new_assignments
    break
  }

  centroids <- new_centroids
  cluster_assignments <- new_assignments
}

return(list(centroids = centroids, cluster_assignments = cluster_assignments))
}

```

Problem 1c

The custom and built-in `k_means` functions may not always have identical results due to arbitrary clustering, however the contingency table shows the same points are joining the same cluster, showing that both the functions have similar results.

```

## write answer here
peng_scaled <- read.csv("peng_scaled.csv")
data_matrix <- as.matrix(peng_scaled)

```

```
# run custom k-means.  
set.seed(100)  
custom_km <- my_kmeans(data_matrix, K = 3, max_iter = 100, tol = 0.001)
```

Converged in 7 iterations.

```
# run built in k-means  
set.seed(100)  
builtin_km <- kmeans(data_matrix, centers = 3, iter.max = 100)  
  
# compare  
comparison_table <- table(custom_km$cluster_assignments, builtin_km$cluster)  
print(comparison_table)
```

	1	2	3
1	64	0	0
2	0	151	0
3	0	0	127